`useContext` in React

`useContext` is a React Hook used to access data from a **Context**.

It helps us share data between components **without passing props manually** through every level.

This solves the problem called:

“Prop Drilling”

What is Prop Drilling?

Imagine:

```
App
 ↓
Parent
 ↓
Child
 ↓
GrandChild
```

If `App` has data and `GrandChild` needs it:

```
<App user="John" />
```

You pass props through every component:

```
Parent → Child → GrandChild
```

Even if middle components do not use it.

This becomes messy.

Solution → Context API + useContext

React provides:

Feature	Purpose
`createContext()`	Creates shared data
`Provider`	Supplies data
`useContext()`	Consumes data

Basic Flow

```
Create Context
      ↓
Provide Context
      ↓
Consume Context using useContext
```

Syntax

Step 1 — Create Context

```
const MyContext = createContext();
```

Step 2 — Provide Context

```
<MyContext.Provider value={data}>
   <App />
</MyContext.Provider>
```

Step 3 — Consume Context

```
const value = useContext(MyContext);
```

Example 1 — Simple User Example

Project Structure

```
src/
 ├── App.js
 ├── UserContext.js
 ├── Parent.js
 ├── Child.js
 └── GrandChild.js
```

Step 1 — Create Context

UserContext.js

```
import { createContext } from 'react';

const UserContext = createContext();

export default UserContext;
```

What Happened Here?

`createContext()`

Creates a global container for data.

Step 2 — Provide Context

App.js

```
import React from 'react';
import UserContext from './UserContext';
import Parent from './Parent';

function App() {

  const user = "John Doe";

  return (

    <UserContext.Provider value={user}>
      <Parent />
    </UserContext.Provider>

  );
}

export default App;
```

Important Part

```
<UserContext.Provider value={user}>
```

This shares data with all child components.

Step 3 — Parent Component

Parent.js

```
import React from 'react';
import Child from './Child';

function Parent() {
  return <Child />;
}

export default Parent;
```

Step 4 — Child Component

Child.js

```
import React from 'react';
import GrandChild from './GrandChild';

function Child() {
  return <GrandChild />;
}

export default Child;
```

Step 5 — Consume Context

GrandChild.js

```
import React, { useContext } from 'react';
import UserContext from './UserContext';

function GrandChild() {

  const user = useContext(UserContext);

  return (
    <h1>Welcome {user}</h1>
  );
}

export default GrandChild;
```

Output

```
Welcome John Doe
```

How `useContext` Works Internally

App Component

```
<UserContext.Provider value="John">
```

Stores value in Context.

GrandChild Component

```
const user = useContext(UserContext);
```

Reads nearest Provider value.

Visual Representation

```
Provider
   ↓
   ↓
   ↓
Consumer
```

Why `useContext` is Useful

Without Context:

```
App
 ↓ user
Parent
 ↓ user
Child
 ↓ user
GrandChild
```

With Context:

```
App (Provider)
 ↓
 ↓
 ↓
GrandChild (Consumer)
```

Cleaner code.

Real-World Use Cases

`useContext` is commonly used for:

Use Case	Example
Authentication	Logged-in user
Theme	Dark/Light mode
Language	English/Hindi
Cart	Shopping cart data
Global Settings	App configurations
Notifications	Toast messages

Example 2 — Dark/Light Theme Toggle

Step 1 — Create Theme Context

ThemeContext.js

```
import { createContext } from 'react';

const ThemeContext = createContext();

export default ThemeContext;
```

Step 2 — Provide Theme

App.js

```
import React, { useState } from 'react';
import ThemeContext from './ThemeContext';
import Home from './Home';

function App() {

  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {

    setTheme(theme === "light" ? "dark" : "light");
  };

  return (

    <ThemeContext.Provider value={{ theme, toggleTheme }}>

      <Home />

    </ThemeContext.Provider>

  );
}

export default App;
```

Here We Passed Object

```
value={{ theme, toggleTheme }}
```

Context can share:

- Strings
- Arrays
- Objects
- Functions
- State
- Anything

Step 3 — Consume Theme

Home.js

```
import React, { useContext } from 'react';
import ThemeContext from './ThemeContext';

function Home() {

  const { theme, toggleTheme } = useContext(ThemeContext);

  return (

    <div
      style={{
        background: theme === "light" ? "white" : "black",
        color: theme === "light" ? "black" : "white",
        height: "100vh"
      }}
    >

      <h1>{theme} mode</h1>

      <button onClick={toggleTheme}>
        Toggle Theme
      </button>

    </div>
  );
}

export default Home;
```

Output

Initially:

```
light mode
```

After button click:

```
dark mode
```

`useContext` + `useReducer`

Very powerful combination.

Used for:

- Global state management
- Redux-like architecture

Example Structure

```
Context
   +
useReducer
   =
Global State Management
```

Example

```
const StoreContext = createContext();
```

Then:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Provide globally:

```
<StoreContext.Provider value={{ state, dispatch }}>
```

Consume anywhere:

```
const { state, dispatch } = useContext(StoreContext);
```

Very common pattern.

Important Rules

---

1. Component Must Be Inside Provider

Wrong:

```
const value = useContext(MyContext);
```

without Provider.

Then value becomes:

```
undefined
```

---

2. Closest Provider Wins

Example:

```
<MyContext.Provider value="A">
   <MyContext.Provider value="B">
      <Child />
   </MyContext.Provider>
</MyContext.Provider>
```

`Child` gets:

```
B
```

Nearest provider value.

---

3. Context Causes Re-render

When Provider value changes:

```
value={theme}
```

All consumers re-render.

Common Mistakes

Mistake 1 — Forgetting Provider

 Wrong

```
useContext(UserContext)
```

without wrapping components.

Mistake 2 — Passing New Object Every Render

 Problematic

```
value={{ user: "John" }}
```

Creates new object each render.

Can cause unnecessary re-renders.

Better

```
const value = useMemo(() => ({ user }), [user]);
```

When NOT to Use Context

Avoid for:

- Frequently changing large states
- Performance-critical apps

Because all consumers re-render.

Sometimes Redux/Zustand/Recoil is better.

`useContext` vs Props

Feature	Props	Context
Pass data manually	Yes	No
Good for small hierarchy	Yes	Yes
Good for deep hierarchy	No	Yes
Avoid prop drilling	No	Yes
Global sharing	Difficult	Easy

Mental Model

Think of Context like:

```
Global Data Storage
```

And `useContext` is:

```
Reading data from storage
```

Simple Analogy

Imagine a house.

Without Context:

```
You carry water bucket room-to-room
```

With Context:

```
House has central water supply
```

Every room accesses directly.

That is exactly what Context does.

Final Summary

`useContext`:

 Avoids prop drilling
 Shares global data easily
 Makes code cleaner
 Works great with `useReducer`
 Perfect for themes/auth/cart/settings

Most Important Interview Question

What problem does `useContext` solve?

Answer:

`useContext` solves prop drilling by allowing components to access shared/global data directly without passing props manually through intermediate components.